

AI Workflows for Empirical Research

General Setup: Claude Code & Codex

Timur Öztürk

University of Bayreuth

July 23, 2026

The instructions file

- ▶ Agentic tools read a Markdown file at every session start — persistent project memory.
- ▶ Codex (+ ~20 tools) call it `AGENTS.md`; Claude Code calls it `CLAUDE.md` (and also reads `AGENTS.md`).
- ▶ Open standard since Aug 2025 (Linux Foundation); 60,000+ repos. Just Markdown, no required fields.
- ▶ Like a “README for the agent”: what you’d tell a new RA, written once.

▶ agents.md

▶ Claude Code memory

Using it well

- ▶ Keep it short (Anthropic: $< \sim 200$ lines); put run/test commands at the top.
- ▶ Sections: overview, setup commands, conventions (e.g. cluster SEs), data-security notes.
- ▶ **MCPs** = connections to external tools/data; **skills** = reusable, on-demand instructions.
- ▶ Both run in a sandbox and can execute code + edit files — power and risk for reproducibility.

▶ AGENTS.md guide

▶ CLAUDE.md best practice

▶ Codex CLI guide

Skills: what they are

- ▶ A folder with a `SKILL.md` (YAML name + description, then Markdown instructions); can bundle scripts + resources.
- ▶ Like an onboarding guide for a repeatable task (e.g. “format regression tables”).
- ▶ **Progressive disclosure**: metadata always loaded (~100 tokens); instructions load on trigger; bundled files/scripts load only when used.
- ▶ The description decides when Claude fires the skill — say *what* it does and *when* to use it.

▶ [Agent Skills docs](#)

Where to find & install them

- ▶ First-party (trusted): Anthropic's repo + pre-built doc skills (pptx, xlsx, docx, pdf).
- ▶ Community directories (vet first): awesome-claude-skills lists, marketplace sites.
- ▶ Install by surface: Claude Code = drop in `~/.claude/skills/` or `.claude/skills/`; claude.ai = upload zip; API = `/v1/skills`.
- ▶ Don't sync across surfaces; API sandbox has no network access.

▶ [anthropics/skills](#)

▶ [awesome-claude-skills](#)

Are they safe?

- ▶ Anthropic: use skills only from trusted sources — treat like installing software.
- ▶ Research: prompt injection in $\sim 36\%$ of skills studied; $\sim 91\%$ of malicious ones bundle malware too.
- ▶ Audit every file (incl. scripts + secondary refs); flag external URL fetches; scope Claude's permissions.
- ▶ Not covered by zero-data-retention — relevant for confidential microdata.

▶ Snyk ToxicSkills

▶ Datadog Security Labs

MCPs in empirical research (I)

What they are, briefly: standardized connectors that let Claude reach your tools and data (databases, files, APIs, reference managers).

- ▶ **Data access:** pull series directly — e.g. FRED MCP exposes 800,000+ economic time series to Claude.
- ▶ **Literature:** search arXiv / OpenAlex / Semantic Scholar and push hits into Zotero for review.
- ▶ **Your project:** filesystem / database servers so Claude reads your data schema and files in place.

▶ FRED MCP

▶ Zotero MCP

▶ MCP Registry

MCPs in empirical research (II)

Workflow & caution:

- ▶ Chain steps: fetch data → clean → estimate → draft — without leaving the agent.
- ▶ Reproducibility: version control + issue trackers via MCP keep an audit trail of what changed.
- ▶ **Security:** a server is code with data + execution access; prefer local servers you control, scope credentials, vet third-party ones.
- ▶ For confidential microdata: avoid remote servers; treat community servers like any unvetted dependency.

▶ Tool poisoning (OWASP)

▶ AI for economists